

Supervised Machine Learning: Classification IBM Skills Network Project Report

**By:
Than Phuc Hau**

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
Main Objective.....	3
Brief Description.....	3
Data cleaning & Feature engineering.....	4
Drop unused column.....	4
Check for any duplicates.....	4
Check for missing value and datatype.....	5
Check for data balance.....	5
Data distribution.....	6
Classifier Models.....	8
Logistic Regression.....	8
Gradient Boosting.....	9
Stacking.....	10
Recommended Model.....	11
Key Findings and Insights.....	11
Suggestions.....	11

Main Objective

As the quality of life has improved significantly in recent years, people have become more aware of their health conditions. With the remarkable development of technology and data-driven healthcare, large-scale medical datasets are now available for analysis. However, not everyone has sufficient knowledge to interpret these data effectively, leading to challenges in assessing personal health risks.

This project aims to predict the presence of cardiovascular disease based on several health indicators.

Brief Description

The data set has many useful information that would help us identify and classify our target. Shown below is a snippet of `data.head()` which shows us a brief about the values of our columns. The data set has **7,000 examples**, the target column is **‘cardio’** being either **0** or **1** and features include:

- id
- age
- gender
- height
- weight
- ap_hi
- ap_lo
- cholesterol
- gluc
- smoke
- alco
- active

	Unnamed: 0	id	age	gender	height	weight	ap_hi	ap_lo	cholesterol	gluc	smoke	alco	active	cardio
0	0	0	50	2	168	62.0	110	80	1	1	0	0	1	0
1	1	1	55	1	156	85.0	140	90	3	1	0	0	1	1
2	2	2	51	1	165	64.0	130	70	3	1	0	0	0	1
3	3	3	48	2	169	82.0	150	100	1	1	0	0	1	1
4	4	4	47	1	156	56.0	100	60	1	1	0	0	0	0

The features are **car_ID**, **symboling**, **CarName**, **fueltype**, **aspiration**, **doornumber**,... with 10 features are categorical and 16 features are numerical.

Some features we will not use such as **car_ID** and **CarName**, we will drop or extract useful details in these columns. We will also encode categorical features to use for the model.

In the next sections, we shall explore more information about the underlying patterns in this data set.

Data cleaning & Feature engineering

Drop unused column

```
df.drop(columns = 'Unnamed: 0', axis = 0, inplace = True)
```

We will drop column “**Unnamed: 0**” as there is no use of it.

Check for any duplicates

```
df.id.is_unique
```

```
True
```

There is no duplicated value.

Check for missing value and datatype

```
df.info()
```

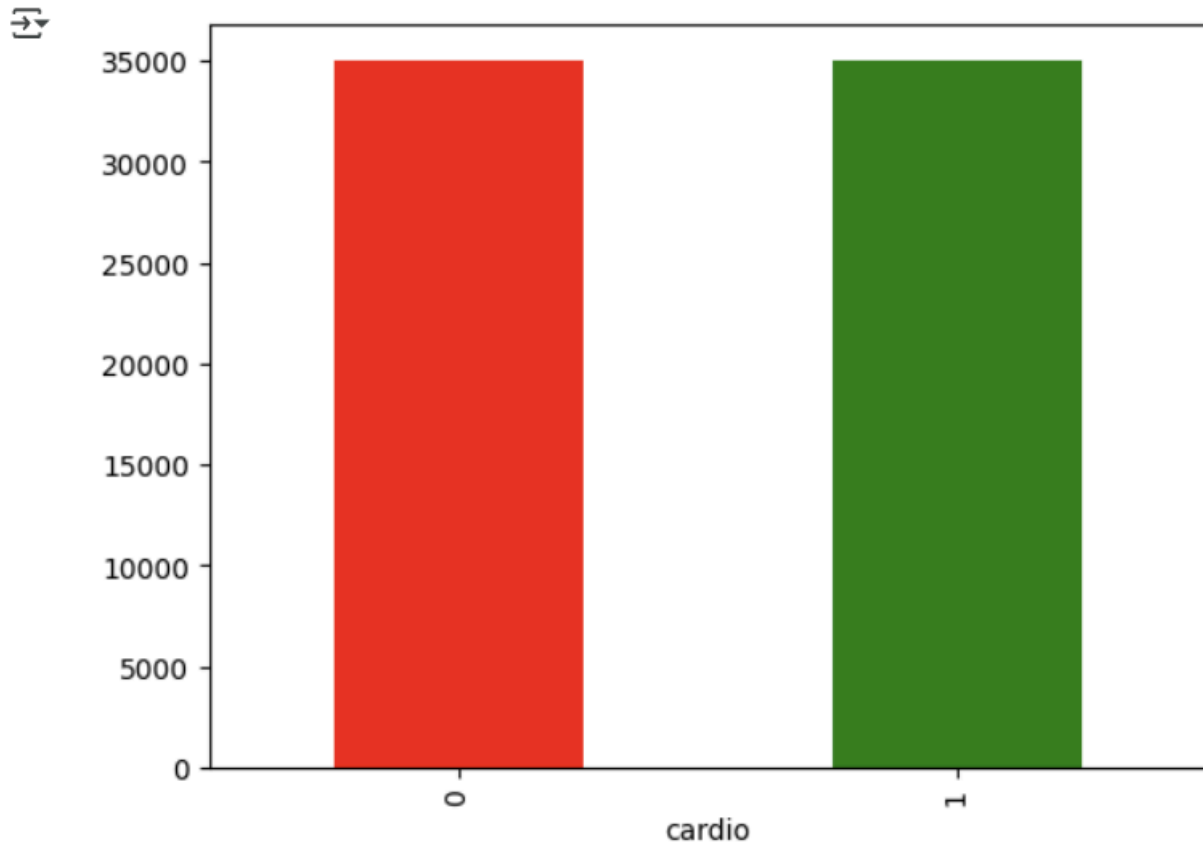
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 70000 entries, 0 to 69999
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   id              70000 non-null  int64
 1   age             70000 non-null  int64
 2   gender          70000 non-null  int64
 3   height          70000 non-null  int64
 4   weight          70000 non-null  float64
 5   ap_hi           70000 non-null  int64
 6   ap_lo           70000 non-null  int64
 7   cholesterol     70000 non-null  int64
 8   gluc            70000 non-null  int64
 9   smoke           70000 non-null  int64
10   alco            70000 non-null  int64
11   active          70000 non-null  int64
12   cardio          70000 non-null  int64
dtypes: float64(1), int64(12)
memory usage: 6.9 MB
```

There is no missing value in these columns, so we will bypass this step.

Check for data balance

We can examine if there exists any imbalance in our target column. We will do that by using a bar plot for our 'cardio' target column. As shown below, we can clearly see that there is no imbalance in our target column, so we need not use any data balancing techniques.

```
import matplotlib.pyplot as plt
y_col = 'cardio'
df[y_col].value_counts().plot.bar(color=['red', 'green'])
plt.show()
```



Data distribution

We also will use **StratifiedShuffleSplit** to ensure good distribution between our train and test sets respectively. We managed to split into 50% **1**s and 50% **0** for both our train and test target variables.

```

from sklearn.model_selection import StratifiedShuffleSplit
strat_shuf_split = StratifiedShuffleSplit(n_splits = 1, test_size = 0.3, random_state = 42)
train_idx, test_idx = next(strat_shuf_split.split(df.drop(y_col, axis=1), df[y_col]))
X_train = df.drop(y_col, axis = 1).iloc[train_idx,:]
Y_train = df[y_col].iloc[train_idx]
X_test = df.drop(y_col, axis = 1).iloc[test_idx,:]
Y_test = df[y_col].iloc[test_idx]

```

```
X_train.shape, Y_train.shape, X_test.shape, Y_test.shape
```

```
((49000, 12), (49000,), (21000, 12), (21000,))
```

```
Y_train.value_counts(normalize = True)
```

proportion

cardio

0	0.500306
---	----------

1	0.499694
---	----------

dtype: float64

```
Y_test.value_counts(normalize = True)
```

proportion

cardio

0	0.500286
---	----------

1	0.499714
---	----------

dtype: float64

Classifier Models

Logistic Regression

We will start with **LogisticRegression** as our base-line model so we can compare the rest to base-line. We will use class weighting too. Our Logistic Regression model did good enough with statistics shown below.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

class_weight = {}
# Assign weight of class 0 to be 0.29
class_weight[0] = 0.29
# Assign weight of class 1 to be 0.71
class_weight[1] = 0.71
model_lr = LogisticRegression (random_state = 123,
                               max_iter = 1000,
                               class_weight = class_weight)

model_lr.fit(X_train, Y_train)
```

```
preds_lr = model_lr.predict(X_test)
```

```
print(classification_report(Y_test, preds_lr))
```

	precision	recall	f1-score	support
0	0.81	0.28	0.42	10506
1	0.56	0.93	0.70	10494
accuracy			0.61	21000
macro avg	0.69	0.61	0.56	21000
weighted avg	0.69	0.61	0.56	21000

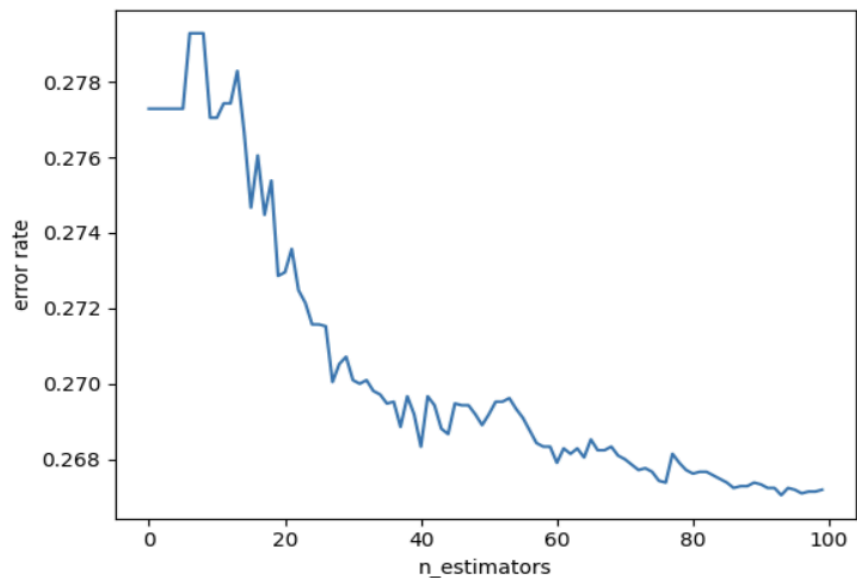
Gradient Boosting

We will now move on to more complex models. For our second model we will use **GradientBoostingClassifier**. We tried fitting our model to several number of estimators to find the best out of all of them. Results hinted towards `n_estimators = 100` being the best number of trees for the GradientBoostingClassifier.

```
print(classification_report(preds_gbc, Y_test))
```

	precision	recall	f1-score	support
0	0.77	0.72	0.74	11261
1	0.70	0.75	0.72	9739
accuracy			0.73	21000
macro avg	0.73	0.73	0.73	21000
weighted avg	0.74	0.73	0.73	21000

```
plt.plot(error_df)
plt.xlabel('n_estimators')
plt.ylabel('error rate')
plt.show()
```

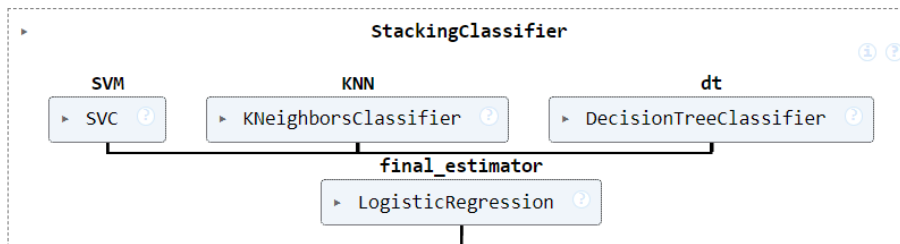


Stacking

We can see from the error plot above that it seems to plateau after 100 estimators so we will be using that. Our model produced outstanding results. For the last model we will be using a **StackingClassifier**, this would incorporate SVC, KNN, and DecisionTreeClassifier and will use a final_estimator of a LogisticRegression model.

```
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
```

```
estimators = [('SVM', SVC(random_state = 42)), ('KNN', KNeighborsClassifier()), ('dt', DecisionTreeClassifier())]
clf = StackingClassifier(estimators = estimators, final_estimator = LogisticRegression())
clf.fit(X_train, Y_train)
```



```
StackingClassifier(estimators = [('SVM', SVC(random_state = 42)), ('KNN', KNeighborsClassifier()), ('dt', DecisionTreeClassifier())],
                    final_estimator = LogisticRegression())
```

```
preds_clf = clf.predict(X_test)
```

```
print(classification_report(preds_clf, Y_test))
```

	precision	recall	f1-score	support
0	0.69	0.65	0.67	11213
1	0.63	0.67	0.65	9787
accuracy			0.66	21000
macro avg	0.66	0.66	0.66	21000
weighted avg	0.66	0.66	0.66	21000

Recommended Model

Based on the statistics shown above we can clearly state that the recommended model is **GradientBoostingClassifier** and this is due to the extraordinary performance demonstrated in the previous section. The model has performed well on all scores with number of estimators of only 100. We choose that model because it is not as complex and still does the needed job and has good scores.

Key Findings and Insights

Our models showed good performance in predicting cardiovascular disease using features like age, gender, blood pressure, cholesterol, and glucose levels. The **StackingClassifier** achieved the best results but tended to **overfit** due to its complexity. The **GradientBoostingClassifier** also performed well, especially on the training data, while **LogisticRegression** gave moderate but more interpretable results. Overall, there is a trade-off between accuracy and interpretability.

Suggestions

To improve the model, we can apply **GridSearchCV** to fine-tune hyperparameters and reduce overfitting. Saving trained models with **pickle** will allow for reuse or further development. Future work could also include adding more health-related features, such as lifestyle and family history, to increase prediction accuracy.